NVIDIA TRITON CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW & SETUP

Installation

NVIDIA Triton AI/ML framework

```
pip install nvidia-triton
# Verify installation
python -c "import nvidia_triton; print('OK')"
# GPU support check
import torch; print(torch.cuda.is_available()...
```

Use virtual environment: python -m venv venv

02. DATA PREPARATION

Architecture

```
import pandas as pd
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1).values
y = df['target'].values
from sklearn.model_selection import train_tes...
X_train, X_test, y_train, y_test = train_test...
```

Normalize/standardize features for best results

03. MODEL DEFINITION

YAML / Config

```
# Simple neural network:
import torch.nn as nn
class Model(nn.Module):
def __init__(self):
super().__init__()
self.layers = nn.Sequential(
nn.Linear(input_size, 128),
nn.ReLU(),
nn.Linear(128, num_classes)
)
def forward(self, x): return self.layers(x)
```

04. TRAINING LOOP

Workflow

```
optimizer = torch.optim.Adam(model.parameters...
criterion = nn.CrossEntropyLoss()
for epoch in range(num_epochs):
for batch_X, batch_y in dataloader:
optimizer.zero_grad()
output = model(batch_X)
loss = criterion(output, batch_y)
loss.backward()
optimizer.step()
```

05. EVALUATION

CRUD API

```
from sklearn.metrics import accuracy_score, c...
model.eval()
with torch.no_grad():
preds = model(X_test)
accuracy = accuracy_score(y_test, preds.argma...
print(classification_report(y_test, preds.arg...
```

Track precision, recall, F1; use confusion matrix

06. INFERENCE & SERVING

Integration

```
# Save model
torch.save(model.state_dict(), 'model.pt')
# Load model
model.load_state_dict(torch.load('model.pt'))
model.eval()
# Inference
with torch.no_grad():
prediction = model(input_tensor)
```

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. HYPERPARAMETERS

Hardening

Learning rate: start with 1e-3, tune with scheduler
Batch size: 32-256 depending on GPU memory
Epochs: use early stopping to prevent overfitting

```
scheduler = torch.optim.lr_scheduler.ReduceLR...
scheduler.step(val_loss)
```

Grid search or Optuna for systematic tuning

08. DATASETS & LOADERS

Observability

```
from torch.utils.data import Dataset, DataLoa...
class MyDataset(Dataset):
def __len__(self): return len(self.data)
def __getitem__(self, idx): return self.data[...
loader = DataLoader(dataset, batch_size=32, s...
```

Pin memory for faster GPU data transfer

09. OPTIMIZATION TECHNIQUES

Optimization

Mixed precision: torch.cuda.amp.autocast()",
Gradient clipping: clip_grad_norm_(model.parameters(), 1.0)",
Batch normalization: nn.BatchNorm1d()",
Dropout: nn.Dropout(0.5) for regularization",

```
# Learning rate warmup
from transformers import get_linear_schedule_...
```

10. FRAMEWORKS & TOOLS

Unit Tests

```
TensorFlow / Keras     production ML
PyTorch                research & NLP
Scikit-learn           classical ML
Hugging Face           pretrained transformers
MLflow / W&B           experiment tracking
ONNX                   model export/portabili...
```

11. DEPLOYMENT

Commands

```
# Export to ONNX
torch.onnx.export(model, dummy_input, 'model....
# FastAPI serving example
@app.post('/predict')
def predict(data: InputData):
tensor = preprocess(data)
return model(tensor).tolist()
```

Use TorchServe, BentoML, or Triton for production

12. COMMON GOTCHAS

Warning

Gradient explosion: use gradient clipping",
Memory leak: always zero_grad() before backward()",
Train/eval mode: model.train() vs model.eval()",
GPU-CPU mismatch: ensure all tensors on same device",
Data leakage: split before normalization, not after",